

Gaussian orthant probabilities with BootLoops

Implementation documentation

(Dated: 5 October 2026)

The solver integrates a multivariate Gaussian density over regions defined by coordinate bounds. These regions include orthants, finite boxes, and boxes that extend to infinity in some directions. For general covariance matrices, it connects the target integral to an independent Gaussian integral, reduces its derivatives to a finite system of boundary integrals, and uses BootLoops to transport that system. Two structured matrix families instead admit one-dimensional quadrature. This guide gives the reduction, the computational steps, and the main limits of accuracy and dimension.

I. BASIC IDEA

The idea of the solver is to convert an integral to a differential equation. We introduce a parameter, differentiate the integral with respect to it, and use integration by parts to eliminate the additional integrals that appear. Known starting values then allow numerical evolution to the target. A Gaussian half-line integral illustrates these steps:

$$J(t) = \int_0^\infty e^{-x^2/2-tx} dx, \quad J(0) = (\pi/2)^{1/2}. \quad (1)$$

Differentiation gives $\frac{dJ}{dt} = -M_1(t)$, where $M_1(t) = \int_0^\infty x e^{-x^2/2-tx} dx$ is an unnormalized first moment. To eliminate it, write $f_t(x) = e^{-x^2/2-tx}$ and integrate $\partial f_t / \partial x = -(x+t)f_t$ over the half-line. The boundary values give $f_t(\infty) - f_t(0) = -1 = -M_1(t) - tJ(t)$, so $M_1(t) = 1 - tJ(t)$. Substitution closes the equation:

$$\frac{dJ}{dt} = tJ(t) - 1, \quad J(0) = (\pi/2)^{1/2}. \quad (2)$$

Solving this first-order ordinary differential equation yields the integral at the desired parameter value. The constant term comes from the finite boundary at $x = 0$; the contribution at infinity vanishes.

This example varies a linear term in the exponent. The general solver instead varies the coupling between Gaussian coordinates. It applies the same reduction to the full integral and its boundary integrals, producing a coupled first-order system that BootLoops evolves from known starting values.

II. PROBABILITY AND CONTINUATION

The integrals the solver actually evaluates are of course more complicated than described above. Let X have mean μ and positive-definite covariance Σ . Set $Q = \Sigma^{-1}$, $x = X - \mu$, $\ell = L - \mu$, and $u = U - \mu$. The target is

$$p = \frac{(\det Q)^{1/2}}{(2\pi)^{d/2}} J_{\text{full}}(1), \quad J_{\text{full}}(t) = \int_{\ell \leq x \leq u} \exp[-x^T Q(t) x / 2] d^d x. \quad (3)$$

The inequalities apply coordinate by coordinate. Define $D = \text{diag}(Q)$ and $E = Q - D$, then set $Q(t) = D + tE$ for $0 \leq t \leq 1$. At $t = 0$ the integral factors into one-dimensional Gaussian integrals; at $t = 1$ it is the requested integral. The path remains valid because $Q(t) = (1-t)D + tQ$ is

positive definite. The solver transports unnormalized integrals and applies the prefactor in Eq. (3) only at the end.

This construction follows the integration-by-parts and differential-equation strategy used for scattering integrals[1–3]. The auxiliary mass flow method supplies another example of transport from a tractable starting point[4]. Here the independent Gaussian at $t = 0$ already supplies that point, so no auxiliary mass is needed. Holonomic gradient methods apply related differential systems to Gaussian orthant probabilities[5].

III. BOUNDARY INTEGRALS AND THE DIFFERENTIAL SYSTEM

Differentiating J_{full} introduces Gaussian moments. The solver eliminates them by integrating by parts and retaining integrals over finite faces of the domain. A face F leaves each coordinate free or fixes it at a finite lower or upper endpoint; infinite endpoints contribute no boundary term. If A contains the free coordinates, B the fixed coordinates, and $b = x_B$, write $J_F(t) = \int_{\ell_A^{u_A}} \exp[-x^\top Q(t)x/2] dx_A$. These are density-slice integrals, not probabilities of landing on a face.

The reduction can be seen from one face. Put $f_t = \exp[-x^\top Q(t)x/2]$, $m_i = \int_F x_i f_t dx_A$, and $M_{ij} = \int_F x_i x_j f_t dx_A$ for free indices. Direct differentiation gives

$$\frac{dJ_F}{dt} = -\frac{1}{2} \text{tr}(E_{AA}M) - (E_{AB}b)^\top m - \frac{1}{2} b^\top E_{BB} b J_F. \quad (4)$$

For a free coordinate j , let Δ_j be the upper child-face integral minus the lower one, omitting infinite endpoints. Integrating $\partial f_t / \partial x_j$ gives $\Delta = -Q_{AA}(t)m - Q_{AB}(t)b J_F$. Integrating $\partial(x_i f_t) / \partial x_j$ similarly expresses M in terms of J_F , m , and child-face terms. A child face fixes one more coordinate, so recursive reduction ends at corner values. Substitution into Eq. (4) yields the closed system

$$\frac{dJ}{dt} = \mathcal{A}(t)J(t). \quad (5)$$

Each entry of $\mathcal{A}(t)$ is rational: the inversions involve principal submatrices of the linear matrix $Q(t)$. The retained family need not be a minimal basis.

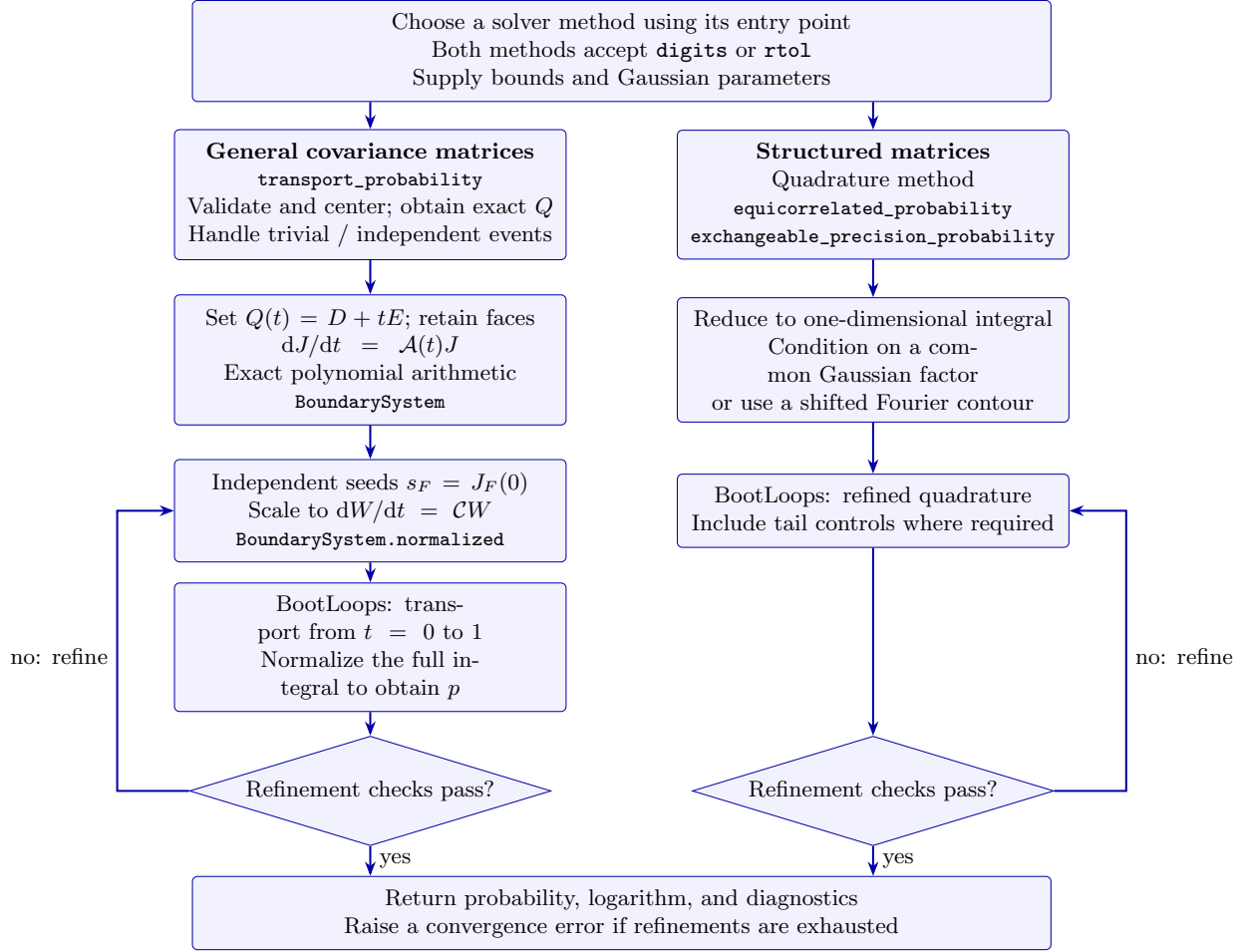
At $t = 0$, a free coordinate i contributes $(2\pi/D_{ii})^{1/2}[\Phi(u_i D_{ii}^{1/2}) - \Phi(\ell_i D_{ii}^{1/2})]$; a coordinate fixed at b_i contributes $\exp(-D_{ii}b_i^2/2)$. Their product gives $s_F = J_F(0)$. Scaling by $S = \text{diag}(s_F)$ gives $W = S^{-1}J$ and

$$\frac{dW}{dt} = \mathcal{C}(t)W, \quad \mathcal{C} = S^{-1}\mathcal{A}S, \quad W(0) = \mathbf{1}. \quad (6)$$

BootLoops advances this rational system from $t = 0$ to $t = 1$ with local Taylor expansions and step control. Finally, $p = (\det Q)^{1/2} s_{\text{full}} W_{\text{full}}(1) / (2\pi)^{d/2}$.

IV. IMPLEMENTATION

Algorithm 1 states the general route. The independent and two structured routes have separate entry points. The flow chart below shows the numerical workflow and the role of each software component.



Algorithm 1 General Gaussian box or orthant probability

Require: Bounds L, U ; mean μ ; covariance Σ or precision Q ; `digits` or `rtol`

Ensure: Probability, logarithm, and diagnostics

- 1: Validate and center the inputs; obtain exact precision Q .
 - 2: **if** the event is empty, full, or independent **then**
 - 3: **return** the corresponding direct result.
 - 4: **end if**
 - 5: Set $Q(t) \leftarrow \text{diag}(Q) + t[Q - \text{diag}(Q)]$.
 - 6: Count and retain eligible faces, merging symmetric ones when possible.
 - 7: Build the rational system $dJ/dt = AJ$ by integration by parts.
 - 8: **for** each permitted precision and transport refinement **do**
 - 9: Evaluate seeds $s_F = J_F(0)$ and form $C = S^{-1}AS$.
 - 10: Use BootLoops to transport $dW/dt = CW$, $W(0) = \mathbf{1}$, to $t = 1$.
 - 11: Normalize the full-region component to obtain p .
 - 12: **if** successive refinements and residual checks pass **then**
 - 13: **return** p , $\log p$, and diagnostics.
 - 14: **end if**
 - 15: **end for**
 - 16: Raise a convergence error.
-

SymPy converts supplied decimal strings and fractions to exact rational inputs, centers the bounds, and constructs the precision matrix and its determinant[6]. The Python connection builder uses SymPy rational expressions; an optional compiled polynomial backend performs the same face

reduction with rational polynomial arithmetic. BootLoops evaluates the system numerically. The adapter passes sparse rational coefficients and estimated pole locations to its transport engine. Poles cannot lie on the real path because all principal submatrices of $Q(t)$ remain positive definite; nearby complex poles may still restrict step sizes.

As a check, take the positive quadrant in two dimensions with $Q(t) = \begin{pmatrix} 1 & tc \\ tc & 1 \end{pmatrix}$. The full integral starts at $J(0) = \pi/2$; its edge and corner integrals also have elementary starting values. Integration by parts gives a rational differential system. With $c = -1/2$, the final covariance has correlation $1/2$, and transport followed by Gaussian normalization gives $p = 1/3$, rather than the independent value $1/4$.

V. STRUCTURED CASES, ACCURACY, AND LIMITS

Two matrix families avoid the face system. For equal nonnegative correlation ρ and common standardized bounds ℓ, u , write $X_i = \rho^{1/2} Z + (1 - \rho)^{1/2} Z_i$ with independent standard-normal variables. Conditioning on $Z = z$ gives the one-dimensional integral $p = \int_{\mathbb{R}} \phi(z) g(z)^d dz$, where $g(z) = \Phi[(u - \rho^{1/2} z)/(1 - \rho)^{1/2}] - \Phi[(\ell - \rho^{1/2} z)/(1 - \rho)^{1/2}]$. For exchangeable precision $Q = aI + b\mathbf{1}\mathbf{1}^T$ with $b \geq 0$ and common finite bounds, a Fourier identity separates the quadratic coupling into one integration variable[7]. The implementation shifts and scales that contour to reduce oscillation. BootLoops performs refined one-dimensional quadrature for both families. These reductions do not extend to arbitrary covariance matrices.

The interfaces accept either `digits` or `rtol`. The first asks for a significant-digit representation; the second targets a relative probability error and starts with cheaper settings. Both compare results at increased working precision and tighter numerical controls. Agreement provides evidence of accuracy, not a proof that every reported digit is correct. Unlike Botev’s minimax tilting estimator, which has sampling uncertainty[8], the solver uses deterministic refinements; neither method acquires ground truth from its own error estimate.

For a general orthant the face system has up to 2^d states; a fully bounded box has up to 3^d . The default 256-state cap therefore limits unstructured problems to eight one-sided or five fully bounded coordinates. Symmetry can merge faces, but it does not remove the generic exponential growth. High-dimensional results from the structured one-dimensional functions should not be read as performance for general covariance matrices.

-
- [1] M. Schwartz, *BootLoops: an LLM-driven toolkit for exact quantitative science*, supplied manuscript (2026), Sec. 3.2; [manuscript](#).
 - [2] K. G. Chetyrkin and F. V. Tkachov, Nucl. Phys. B **192**, 159 (1981).
 - [3] A. V. Kotikov, Phys. Lett. B **254**, 158 (1991).
 - [4] X. Liu and Y.-Q. Ma, Comput. Phys. Commun. **283**, 108565 (2023); [arXiv:2201.11669](#).
 - [5] T. Koyama and A. Takemura, *Calculation of orthant probabilities by the holonomic gradient method* (2012); [arXiv:1211.6822](#).
 - [6] A. Meurer *et al.*, PeerJ Comput. Sci. **3**, e103 (2017).
 - [7] J. Hubbard, Phys. Rev. Lett. **3**, 77 (1959).
 - [8] Z. I. Botev, J. R. Stat. Soc. B **79**, 125 (2017); [arXiv:1603.04166](#).